

PHASE 2: REQUIREMENT ANALYSIS

SOLUTION REQUIREMENTS

Date	28 February 2026
Team ID	LTVIP2026TMIDS76950
Team Size	4
Team Leader	Suresh Kumar V
Team Members	Vemanaboina Lakshmi Prasanna Chengalaraya Akhil Prasad Chittipothula
Project Name	HouseHunt: Finding Your Perfect Rental Home

1. FUNCTIONAL REQUIREMENTS

1.1 USER MANAGEMENT

FR-1.1.1: User Registration

- System shall allow users to register as Admin, Owner, or Renter
- Required fields: Name, Email, Password, User Type
- Email must be unique across the system
- Password must be minimum 6 characters
- Password shall be encrypted before storage

FR-1.1.2: User Login

- System shall authenticate users with email and password
- System shall generate JWT token upon successful login
- Token validity: 24 hours
- System shall redirect users to appropriate dashboard based on type

FR-1.1.3: Session Management

- System shall maintain user session using sessionStorage
- Session shall expire on browser close
- System shall clear session data on logout

1.2 ADMIN FUNCTIONALITIES

FR-1.2.1: View All Users

- Admin shall view complete list of registered users

- Display: User ID, Name, Email, Type, Status
- System shall show owner approval status (granted/ungranted)

FR-1.2.2: Owner Approval

- Admin shall approve/reject property owner accounts
- System shall change status from "ungranted" to "granted"
- Approved owners shall gain access to owner dashboard
- System shall allow revoking owner access

FR-1.2.3: View All Properties

- Admin shall view all properties across all owners
- Display: Property ID, Type, Address, Owner, Amount, Availability

FR-1.2.4: View All Bookings

- Admin shall view all booking requests in the system
- Display: Booking ID, Property ID, Renter Name, Status

1.3 OWNER FUNCTIONALITIES

FR-1.3.1: Add Property

- Owner shall add new property listings
- Required fields:
 - Property Type (Residential/Commercial/Land-Plot)
 - Ad Type (Rent/Sale)
 - Address
 - Owner Contact Number
 - Property Amount
 - Property Images (multiple)
 - Additional Information (optional)
- System shall automatically set isAvailable = "Available"
- System shall associate property with owner ID

FR-1.3.2: Edit Property

- Owner shall edit own property details
- Editable fields: All property information
- System shall allow changing property image
- System shall maintain property ownership

FR-1.3.3: Delete Property

- Owner shall delete own properties
- System shall show confirmation dialog
- Deletion shall be permanent
- System shall remove associated bookings

FR-1.3.4: View Own Properties

- Owner shall view list of own properties only
- Display in tabular format with action buttons

FR-1.3.5: View Bookings

- Owner shall view booking requests for own properties
- Display: Renter name, phone, booking status

1.4 RENTER FUNCTIONALITIES

FR-1.4.1: Browse Properties

- Renter shall view all available properties
- Display as cards with images
- Show: Image, Location, Type, Ad Type, Amount

FR-1.4.2: Filter Properties

- Renter shall filter properties by:
 - Address/Location (text search)
 - Property Type (Residential/Commercial/Land-Plot)
 - Ad Type (Rent/Sale)
- Filters shall work in combination
- System shall show real-time filtered results

FR-1.4.3: View Property Details

- Renter shall click "Get Info" to view full details
- Display in modal:
 - Image carousel (if multiple images)
 - Complete property information
 - Owner contact number
 - Booking form

FR-1.4.4: Book Property

- Renter shall fill booking form with:
 - Full Name
 - Phone Number
- System shall create booking record with status "pending"
- System shall link booking to property and owner
- System shall show success message

FR-1.4.5: View Booking History

- Renter shall view own booking requests
 - Display: Booking ID, Property ID, Status
-

2. NON-FUNCTIONAL REQUIREMENTS

2.1 PERFORMANCE REQUIREMENTS

NFR-2.1.1: Response Time

- Page load time: < 2 seconds
- API response time: < 500 milliseconds
- Image load time: < 3 seconds

NFR-2.1.2: Throughput

- System shall handle 100 concurrent users
- System shall process 1000 requests per minute

NFR-2.1.3: Resource Usage

- Frontend bundle size: < 5MB
- Database query time: < 100ms
- Memory usage: < 512MB

2.2 SECURITY REQUIREMENTS

NFR-2.2.1: Authentication

- All passwords shall be hashed using bcrypt (10 salt rounds)
- JWT tokens shall be used for authentication
- Tokens shall expire after 24 hours
- Failed login attempts shall be logged

NFR-2.2.2: Authorization

- Role-based access control shall be enforced

- Users shall only access their authorized resources
- Unauthorized access attempts shall redirect to appropriate page
- API endpoints shall validate user permissions

NFR-2.2.3: Data Protection

- Sensitive data shall be encrypted in transit
- Database credentials shall be stored in environment variables
- User passwords shall never be logged
- File uploads shall be validated for type and size

NFR-2.2.4: Session Security

- Sessions shall use sessionStorage (not localStorage)
- Sessions shall clear on browser close
- Tokens shall be validated on every protected request

2.3 USABILITY REQUIREMENTS

NFR-2.3.1: User Interface

- Interface shall be intuitive and self-explanatory
- Maximum 3 clicks to reach any feature
- Consistent design across all pages
- Clear error messages for user actions

NFR-2.3.2: Accessibility

- Responsive design for all screen sizes
- Support for keyboard navigation
- Readable font sizes (minimum 14px)
- Color contrast ratio > 4.5:1

NFR-2.3.3: Learning Curve

- New users shall complete registration in < 2 minutes
- Property listing shall take < 5 minutes
- Booking shall take < 1 minute
- No training required for basic operations

2.4 RELIABILITY REQUIREMENTS

NFR-2.4.1: Availability

- System uptime: 99% (MVP target)

- Maximum planned downtime: 1 hour per month
- Error recovery: < 5 minutes

NFR-2.4.2: Error Handling

- All API errors shall return meaningful messages
- Frontend shall display user-friendly error messages
- Critical errors shall be logged
- System shall recover gracefully from errors

NFR-2.4.3: Data Integrity

- Database constraints shall prevent invalid data
- Transactions shall be ACID compliant
- Data backups shall be automatic
- Data loss tolerance: 0%

2.5 SCALABILITY REQUIREMENTS

NFR-2.5.1: Horizontal Scaling

- Architecture shall support horizontal scaling
- Database shall support sharding
- Stateless backend design

NFR-2.5.2: Vertical Scaling

- System shall support increased resources
- Code shall be optimized for performance
- Database queries shall be optimized

NFR-2.5.3: Growth Capacity

- Support 10,000 users in Phase 1
- Support 100,000 properties in Phase 1
- Support 1,000,000 bookings in Phase 1

2.6 MAINTAINABILITY REQUIREMENTS

NFR-2.6.1: Code Quality

- Code shall follow consistent style guide
- Functions shall be modular and reusable
- Code comments for complex logic
- Meaningful variable and function names

NFR-2.6.2: Documentation

- API documentation shall be maintained
- User manual shall be provided
- Technical documentation for deployment
- Code-level documentation for developers

NFR-2.6.3: Testing

- Critical paths shall have test coverage
- Bug fixing turnaround: < 24 hours
- New features shall be tested before deployment

2.7 COMPATIBILITY REQUIREMENTS

NFR-2.7.1: Browser Support

- Chrome (latest 2 versions)
- Firefox (latest 2 versions)
- Safari (latest 2 versions)
- Edge (latest 2 versions)

NFR-2.7.2: Device Support

- Desktop (Windows, macOS, Linux)
- Tablet (iPad, Android tablets)
- Mobile (iOS, Android) - responsive design

NFR-2.7.3: Platform Support

- Backend: Any OS supporting Node.js
- Database: MongoDB 4.0+
- Frontend: Modern browsers with JavaScript enabled

3. SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

Development Environment:

- Processor: Intel Core i3 or equivalent
- RAM: 4GB minimum, 8GB recommended
- Storage: 10GB free space
- Network: Broadband internet

Production Server:

- Cloud-based (Heroku/AWS)
- Autoscaling enabled
- Load balancer
- CDN for static files

3.2 SOFTWARE REQUIREMENTS**Development:**

- Node.js v14+ LTS
- npm v6+
- MongoDB 4.0+ (local or Atlas)
- VS Code or any IDE
- Git for version control
- Modern web browser

Production:

- Node.js v14+ LTS
- MongoDB Atlas
- Cloud hosting platform
- SSL certificate
- Domain name

4. DATA REQUIREMENTS**4.1 DATA STORAGE****User Data:**

- Personal information (name, email)
- Encrypted passwords
- User type and status
- Registration timestamps

Property Data:

- Property details (type, address, amount)
- Owner information
- Images (file paths)

- Availability status

Booking Data:

- Renter and owner IDs
- Property reference
- Contact information
- Booking status

4.2 DATA VALIDATION

Input Validation:

- Email format validation
- Phone number format (10 digits)
- Required field checks
- Data type validation
- File type validation (images only)

Business Logic Validation:

- Unique email constraint
- Owner must be approved before property listing
- Valid property ID for bookings
- User authorization for operations

4.3 DATA RETENTION

Active Data:

- User accounts: Indefinite (until user deletion)
- Active properties: Until owner deletes
- Active bookings: Until completion

Archived Data:

- Completed bookings: 1 year
- Deleted properties: 30 days (soft delete)
- User audit logs: 6 months

5. INTERFACE REQUIREMENTS

5.1 USER INTERFACE

Layout:

- Navigation bar (top)
- Main content area
- Footer (optional)
- Responsive grid system

Components:

- Forms for data input
- Tables for data display
- Cards for visual content
- Modals for details
- Notifications for feedback

Navigation:

- Clear menu structure
- Breadcrumbs (where applicable)
- Back buttons
- Home button always accessible

5.2 API INTERFACE

REST API Standards:

- HTTP methods: GET, POST, PUT/PATCH, DELETE
- JSON request/response format
- Status codes: 200, 401, 404, 500
- Bearer token authentication

Endpoints Required:

POST /api/user/register

POST /api/user/login

GET /api/user/getAllProperties

POST /api/user/bookinghandle/:id

GET /api/user/getallbookings

POST /api/owner/postproperty

GET /api/owner/getallproperties

PATCH /api/owner/updateproperty/:id

DELETE /api/owner/deleteproperty/:id

GET /api/admin/getallusers

POST /api/admin/handlestatus

GET /api/admin/getallproperties

GET /api/admin/getallbookings

5.3 DATABASE INTERFACE

MongoDB Collections:

- users
- properties
- bookings

Connection:

- Mongoose ODM
 - Connection pooling
 - Auto-reconnect on failure
-

6. BUSINESS REQUIREMENTS

6.1 USER ACQUISITION

Target:

- 100 users in first month
- 50 properties listed
- 20 successful bookings

Strategy:

- Word-of-mouth marketing
- Social media presence
- Quality listings (admin approval)

6.2 USER RETENTION

Targets:

- 70% user retention after 3 months
- 80% owner satisfaction
- 85% renter satisfaction

Strategy:

- Quality control (admin approval)
- Responsive support
- Regular platform updates

6.3 BUSINESS RULES**BR-1: Owner Approval**

- All owners must be approved by admin before accessing platform

BR-2: Property Ownership

- Owners can only edit/delete own properties

BR-3: Booking Rules

- Renters must be logged in to book
- One booking per renter per property at a time

BR-4: Data Visibility

- Renters see all properties
 - Owners see only own properties
 - Admin sees all data
-

7. CONSTRAINTS**7.1 TECHNICAL CONSTRAINTS**

- Development time: 8 weeks
- Single developer team
- Free tier hosting limitations
- No payment gateway (MVP)

7.2 BUSINESS CONSTRAINTS

- Zero marketing budget
- No dedicated support team
- Manual admin approval process
- Limited to rental properties only

7.3 REGULATORY CONSTRAINTS

- Comply with data protection laws
- User privacy protection

- Terms of service required
 - Cookie policy required
-

8. ASSUMPTIONS

1. Users have stable internet connection
 2. Users have modern web browsers
 3. MongoDB Atlas free tier sufficient for MVP
 4. Admin available for daily approvals
 5. Users provide accurate information
 6. Image sizes reasonable (<5MB)
 7. English language sufficient initially
-

9. DEPENDENCIES

9.1 EXTERNAL DEPENDENCIES

- MongoDB Atlas availability
- npm package availability
- Hosting platform uptime
- Internet connectivity

9.2 INTERNAL DEPENDENCIES

- Admin must approve owners before they can list
 - Properties must exist before bookings
 - Users must be registered to access features
-

10. ACCEPTANCE CRITERIA

10.1 FUNCTIONAL ACCEPTANCE

User can register with all three types
User can login and access appropriate dashboard
Admin can approve/reject owners
Owner can add/edit/delete properties
Renter can browse and book properties
All CRUD operations work correctly
Security measures implemented
Session management works

10.2 NON-FUNCTIONAL ACCEPTANCE

Page load time < 2 seconds
Zero critical security vulnerabilities
99% uptime during testing
Responsive on all devices
No data loss during operations
Error handling works

10.3 USER ACCEPTANCE

Users can complete tasks without help
Interface is intuitive
Feedback is clear
No major bugs
Meets user expectations

11. REQUIREMENT TRACEABILITY

Requirement ID	Priority	Status	Test Case
FR-1.1.1	P0	Complete	TC-001
FR-1.1.2	P0	Complete	TC-002
FR-1.2.2	P0	Complete	TC-003
FR-1.3.1	P0	Complete	TC-004
FR-1.4.4	P0	Complete	TC-005

Document Status: Complete

Date: 28 February 2026

Part of HouseHunt Documentation - Smart Bridge Virtual Internship